

Development Report — CaseGame

Development Report — CaseGame

Panteon Games — Game Developer Strategy Game Demo Unity 2021.3.45f2 (LTS) · Universal Render Pipeline 12.1.15 · Windows standalone, 2D

1. Project Overview

CaseGame is a 2D top-down strategy demo built against Panteon's Game Developer case brief (Docs/Brief/Panteon_GameDeveloperDemoProject2026.pdf). The brief asks for a small RTS slice — building placement, unit production, and unit movement/combat on a grid — implemented with a named set of engineering concepts (OOP, SOLID, Factory, Singleton, MVC, Object Pooling, Coroutines, Events, custom A* pathfinding) and evaluated as much on draw-call budget, code legibility, and project layout as on functional correctness.

The finished project delivers:

- A grid-based Game Board with building placement (valid/invalid visual feedback), Barracks and Power Plant buildings, and three soldier types with distinct attack damage.
- Left-click select / right-click move-or-attack unit control, with custom 8-directional A* pathfinding that routes around buildings.
- An infinite, object-pooled Production Menu and a reactive Information Panel.
- A Main Menu with Play and a resolution/display-mode Settings screen.
- Combat, health, and destruction for both units and buildings.
- A measured, sub-budget render cost (see §4, requirement 13) achieved through an actual architectural change (terrain baking), not configuration alone.
- A playable Windows build, produced and confirmed working outside the repository, shipped alongside the source (see §8).

Forty feature passes (Docs/Reports/001 – 040) plus one closing pre-submission audit built the project from an empty Unity project to this state. This document distills that history for a reader who was not present for it.

2. Architecture Summary

The project is data-driven and MVC-flavored: **Models** are ScriptableObject definitions (`BuildingDefinition` , `UnitDefinition`) and plain C# state (`Health`); **Views** are MonoBehaviours that only render; **Controllers** are MonoBehaviours that translate input and model changes into view updates. Modules communicate through C# events and ScriptableObject event channels rather than direct references, so no two gameplay modules need to know about each other's concrete types.

Module	Responsibility
Core	<code>GameManager</code> — the brief-mandated Singleton, isolated behind <code>IGameManager</code> ; scene lifecycle.
Grid	<code>GridDefinition</code> (designer-editable cell size/extents/terrain margin), <code>GridModel</code> (world↔cell conversion, occupancy, footprints), <code>GridView</code> (runtime grid-line rendering).
Entities	<code>GameEntityDefinition</code> / <code>GameEntityBase</code> — shared base for everything with HP, a sprite, and a death path. Buildings and Units both extend this rather than duplicating it.
Buildings	<code>BuildingBase</code> / <code>Barracks</code> / <code>PowerPlant</code> , <code>BuildingDefinition</code> , <code>BuildingFactory</code> , the building catalog and its event channels.
Units	<code>SoldierBase</code> / <code>Soldier</code> , <code>UnitDefinition</code> , <code>UnitFactory</code> , <code>Projectile</code> / <code>ProjectileFactory</code> (ranged attacks), production request plumbing.
Combat	<code>IDamageable</code> , <code>Health</code> — HP, damage, death, as plain per-instance C# events.
Pathfinding	<code>ASTarPathfinder</code> — static, 8-directional grid A* with corner-cutting prevention; <code>FindApproachCell</code> for "walk into range."
Placement	<code>BuildingGhostView</code> / <code>PlacementController</code> — valid/invalid ghost preview, commit/cancel/remove.
Selection	<code>SelectionController</code> — left-click select (+shift-click multi-select), right-click move-or-attack.
Gameplay	<code>GameplayBootstrap</code> — the one composition root that wires every controller into <code>Gameplay.unity</code> .
CameraControl	<code>CameraController</code> — pan/zoom, clamped to the terrain bounds.
Environment	Procedural island terrain (<code>IslandTileSet</code> / <code>IslandTilemapLayout</code> / <code>IslandTerrainView</code>) baked into a single draw call at load (<code>TerrainCompositor</code>).
Pooling	Generic <code>PrefabPool<T></code> , used by the scroll view, buildings, units, and projectiles.
Events	ScriptableObject event channels (<code>GameEventChannel<T></code>) connecting modules without direct references.
UI.Production / UI.Info / UI.MainMenu / UI.Settings	The Production Menu (pooled infinite scroll), Information Panel, Main Menu, and Settings screen.

Data flow. Building/unit stats live on ScriptableObject definitions; a designer adds a new type by adding an asset and, at most, a thin subclass — no system iterates a hardcoded list of types. Creation goes through Factories, all backed by the same generic pool. Selection raises an event the Information Panel reacts to; production requests, building removal, and combat death all follow the same publish/subscribe shape. No module holds a direct reference to a module it doesn't structurally depend on (enforced in practice — see the code-quality audit in §7).

Rendering. Every entity sprite shares one material and is packed into one of two `SpriteAtlas` assets (gameplay art, UI art), the precondition for Unity's sprite batching. The board's terrain — three Tilemap layers that cannot join the SRP Batchers regardless of configuration — is baked into a single texture on one quad at scene load (§5). Grid lines are one combined mesh, not one renderer per line.

3. Development Timeline

Forty reports, condensed. Each shipped independently with its own compile check, test run, and either automated coverage or an explicit hand-test note.

Foundations (001–011). Grid data model and world/cell math; the Singleton `GameManager` (plus a same-day root-persistence bugfix); a reusable Scene-asset picker (`SceneReference`), adopted project-wide in place of raw string fields; the Main Menu and Settings screen (rebuilt once, mid-project, onto TextMeshPro and the Input System once those were installed); the SO event-channel foundation; a generic object pool; `Health / IDamageable` ; A* pathfinding with corner-cutting prevention. Each landed with no consumer yet — deliberately, to avoid speculative scope.

Core gameplay (009–017). Buildings and Units, sharing a common `GameEntityBase` extracted once the duplication between them became real. Placement (desaturate-then-tint ghost preview, hand-authored shader). The Production Menu's pooled infinite scroll and its recycling math. Selection (click/shift-click, hit-testing via colliders). The Information Panel. Gameplay scene assembly — the integration pass that wired every standalone module into one scene and, in the process, an audit before touching the scene surfaced that units had no actual path to being produced, plus two stale data bugs on shipped definition assets. All three were fixed as part of assembly, not deferred.

Systems hardening (018–028). Draw-call architecture (sprite atlasing, GPU instancing) split from its numeric verification, deferred until visual polish stabilized. Camera pan/zoom. A cluster of architecture fixes: center-anchored footprint placement, a building-removal event path independent of the death/HP path, unit spawn-cell occupancy. A full combat overhaul — attack range, attack speed, sustained auto-attack, move-into-range, and a tracked ranged projectile — replacing the original instant-hit placeholder. A bugfix pass for footprint-aware attack targeting, projectile facing, and stretched UI icons.

Visual polish (029–039). Runtime grid-line rendering as one mesh. Decorative terrain: a static water backdrop and (after the pack's autotile grass/cliff sheet was found to need visual verification the environment couldn't provide) a deliberately simple first pass, later replaced by a procedurally-generated island once the human hand-painted a reference. Camera bounds clamped to the terrain. Unit animations

(idle/run/attack), built by hand-authoring the state machine onto Animator Controllers that shipped as unwired scaffolds. A round of human-flagged polish fixes: unit facing, an Archer release-on-animation-event, a placement/selection input-race bug (fixed twice — the first fix was itself racy, diagnosed and corrected). UI reskinning onto purchased pack art. Damage/death particle effects.

Draw-call optimization (backlog item 20, branch experiment). Once polish was in, the project was split across two branches to measure a real trade-off: `VisualPolish` (everything above, measured at or just over the 20-SetPass budget under heavy combat) versus `Optimisation` (aggressive cuts for headroom). The terrain-baking change described in §5 was `Optimisation`'s one landed cut, brought the measured cost to roughly 10 SetPass calls, and was merged into `main` as the shipping direction.

Pre-submission audit (2026-07-23). A closing pass, not a feature: three independent checks against the brief's literal requirements, code quality/dead code, and project hygiene (scene structure, folder layout, orphaned files). Findings were fixed in the same pass — see §7.

4. Brief Requirements Checklist

Numbers follow `BRIEF.md`'s enumeration of the brief's "GENERAL INFORMATION" section; the brief's own GI-numbers are noted where they differ.

#	Requirement	Status	Where
1	Unity 2021 LTS, 2D, Windows build included in submission	Done	Editor pinned to <code>2021.3.45f2</code> ; Windows build produced and confirmed working (see §8) — shipped as a separate artifact alongside the repository, not committed into it
2	Barracks / Power Plant / Soldier Units in the Production Menu, extensible to future types	Done	<code>BuildingCatalog_Default.asset</code> ; <code>ProductionMenuController</code> iterates it generically, no per-type branch anywhere
3	Placement at a user-chosen location; invalid-location feedback; name/image/dimensions	Done	<code>PlacementController</code> / <code>BuildingGhostView</code> (red/green tint); footprints/cell size are designer-editable fields, matching the brief's own mockup values exactly (Barracks 4×4, Power Plant 2×3, Soldier 1×1)
4	Unlimited, instant production, no cost	Done	<code>BuildingFactory</code> / <code>UnitFactory.Create</code> — no timers, no cost fields anywhere in either definition type
5	Selecting a building shows its image, and its producible units' images, on the Information Panel	Done	<code>InfoPanelController</code>
6	Only Barracks produce units; Power Plant needs no production sub-menu	Done	Falls out of an empty <code>ProducibleUnits</code> list — no special case

#	Requirement	Status	Where
7	Soldiers spawn at a per-barracks spawn point	Done	<code>Barracks.SpawnPosition</code> override
8	Left-click select, right-click move via shortest path, routing around buildings	Done	<code>SelectionController</code> + <code>AStarPathfinder</code> (8-directional, corner-cutting prevented)
9	3 soldier types, 10 HP each, damage 10/5/2	Done, exact match	<code>UnitDef_Soldier1/2/3.asset</code>
10	Building HP: Barracks 100, Power Plant 50	Done, exact match	<code>BuildingDef_Barracks/PowerPlant.asset</code>
11	Selected soldier(s) attack via right-click on a unit or building	Done	<code>SoldierBase.Attack</code> , dispatched from <code>SelectionController</code>
12	Units/buildings destroyed at 0 HP	Done	<code>Health</code> → death event → pool release
13	Draw calls (SetPass calls) under 20, via batching/GPU instancing	Done, measured	Sprite atlasing + shared materials + GPU instancing (Report 018); terrain baked to one draw (Report 040). Measured at 10 SetPass / 12 draw calls in an active-combat scene (Profiler screenshot on file). One accepted exception: a one-time ~25-SetPass spike on the scene's first rendered frame — see §5
14	Works across aspect ratios/resolutions	Done	Canvas in Scale-With-Screen-Size mode; Settings screen demonstrates live resolution/fullscreen switching
15	Three-area interface: Production Menu (infinite scroll, pooled), Game Board, Information Panel	Done	<code>ProductionMenuController</code> / <code>ScrollRecycler</code> (pool never grows with catalog size — confirmed by test), Game Board, <code>InfoPanelController</code>
16	Legible, standards-compliant code	Done	Consistent namespacing mirroring folders; humble-MonoBehaviour/plain-C# split throughout; every code comment audited and rewritten for clarity as a closing pass
17	Scene structure, GameObject naming, folder structure reviewed	Done	Reviewed and corrected in the pre-submission audit (scene-organizer consistency, orphaned files removed, folder-structure documentation brought in line with the actual layout)
18	Edge cases considered	Done	Documented throughout — pooled-instance staleness (selection, event channels), click-ordering races (placement vs. selection, fixed twice), self-targeting attacks, corner-cutting in pathfinding, and others (§5)
19	Main Menu with Play and a Settings screen	Done	<code>MainMenuController</code> / <code>SettingsController</code> ; human-mandated addition to demonstrate requirement 14 directly

Mandated systems/patterns — all demonstrably used, not merely present in name:

Pattern	Where
OOP — Polymorphism, Inheritance	<code>GameEntityBase</code> → <code>BuildingBase/SoldierBase</code> , virtual overrides genuinely used (<code>SpawnPosition</code> , <code>GetNearestOccupiedCell</code>)
SOLID	Open/closed enforced in practice — no type-switches in UI or Factories; single-responsibility humble <code>MonoBehaviours</code> throughout
Factory	<code>BuildingFactory</code> , <code>UnitFactory</code> , <code>ProjectileFactory</code> — all pool-backed
Singleton	<code>GameManager</code> , isolated behind <code>IGameManager</code>
MVC	See §2
Draw Call optimization	See requirement 13
Object Pooling	<code>PrefabPool<T></code> — scroll view rows, buildings, units, projectiles
Coroutines	Unit movement and attack sequencing (<code>SoldierBase</code>)
Events	SO event channels (selection, production, removal) + plain C# events (<code>Health</code>)
Platform: 2D	Orthographic camera, <code>SpriteRenderer</code> -based entities and terrain
Algorithm: A*	<code>AStarPathfinder</code> — custom, grid-based, no <code>NavMesh</code> anywhere in the codebase

5. Decisions & Deviations

Full reasoning for every entry below lives in `ARCHITECTURE.md` 's decisions log; this section pulls out the ones that matter for understanding *why* the shipped system looks the way it does, grouped by cause.

Brief-mandated deviations from general best practice

- **Singleton for `GameManager`** . Generally an anti-pattern for testability and hidden global state; used anyway because the brief names it explicitly as a required pattern. Mitigated by isolating all access behind `IGameManager` rather than letting the concrete type spread through the codebase.
- **No enemy AI / opposing faction, no resource economy.** Both are common RTS staples; both are explicitly out of scope — the brief specifies unlimited, free, instant production, and combat as a demonstrable right-click mechanic against any legal target, not a win/lose battle system.

Interpretive calls where the brief was silent

- **Multi-select via shift-click, not a drag-marquee box.** The brief says "unit(s)" but doesn't specify the mechanism. Shift-click needs no new screen-to-world geometry or selection-rectangle visual and keeps the entire feature unit-testable; a drag-box remains a straightforward follow-up if wanted. A full-featured RTS would very likely want the drag-box regardless (it's the expected genre convention) — shift-click was the right call for a demo prioritizing a fully-tested, low-risk feature over matching genre convention exactly.

- **Combat overhaul beyond the brief's literal minimum.** The brief only requires a right-click attack; the shipped system adds attack range, attack speed, sustained auto-attack, and move-into-range-then-attack, plus a tracked ranged projectile for the Archer-type soldier. This was a deliberate scope expansion, not a misreading — it makes ranged vs. melee combat visibly different rather than both resolving as an identical instant hit.
- **Range/approach checks are not footprint-aware for building targets.** Buildings cannot be referenced from Units (Buildings already depends on Units, for producible-unit data — the reverse would be circular), so an attacking soldier's range check uses a `GameEntityBase` virtual method (`GetNearestOccupiedCell`) rather than reaching into `BuildingBase` directly. Large-footprint buildings may need a generously configured attack range; this is a scoped, documented boundary, not an oversight. A production project with genuinely large multi-cell structures would want true nearest-footprint-edge distance for every ranged interaction, not just the melee-approach case this project already handles — reasonable to defer here since the only shipped buildings (4×4, 2×3) don't make the difference visually obvious.

The draw-call budget and the terrain bake (the clearest example of a deliberate, justified deviation)

Requirement 13 sets a hard budget: draw calls (SetPass calls) under 20. Mid-project measurement on the fully visually-polished build sat right at or just over that line under heavy combat. The single largest cost was the board's terrain: three Tilemap layers that, regardless of shared material or atlas configuration, cannot join Unity's SRP Batcher and additionally split into internal per-chunk draws once the painted area exceeds one chunk (it does, at this grid's size).

The fix baked all three Tilemap layers into one texture on a single sprite quad at scene load, then hid the source Tilemaps — collapsing a guaranteed multi-draw, non-batchable render path into one fully batchable draw, at full art fidelity (a bake, not a downgrade to a placeholder color). Measured result: roughly 10 SetPass calls in active combat, comfortably inside budget.

The bake itself costs one synchronous camera render at the moment it runs, which measurably spikes SetPass calls to roughly 25 on the exact frame it happens — the *first* rendered frame of `Gameplay.unity`, before the Production Menu is even interactive. Three ways to eliminate this were evaluated (deferring the bake by a frame, spreading it across several frames, or moving it to edit time as a precomputed asset) and deliberately not implemented. The reasoning: requirement 13's budget is a steady-state gameplay metric, and in a real production build this exact cost is what a loading screen exists to absorb — gate the reveal of gameplay on the bake's completion, and the spike happens entirely behind loading UI where no draw-call budget is being measured. This project has no loading screen (not brief-mandated; scene transitions are direct), so the spike is visible in isolation if profiled at that one specific frame, but is a one-time, pre-interactive cost with a known, standard production-engineering answer, not an unresolved problem. Building any of the three mitigations into this project specifically — a coroutine-deferred bake, a multi-frame split, or an edit-time precompute pipeline with its own re-bake tooling — would be solving a problem this fixed-scope demo doesn't actually have (there is no loading screen to hide behind, and none

is warranted for a single-scene, direct-transition demo); each would add real code and maintenance surface to eliminate a cost that only ever shows up once, on a frame nothing is measuring. Correct call for a shipping product with a loading sequence already in place; over-engineering here.

Process decision: branch-based A/B measurement

Rather than committing unilaterally to how far visual polish should be cut back for draw-call headroom, the project was split into two branches at the same commit — one keeping full visual polish (measuring at/near the budget line), one applying the terrain-bake cut (measuring comfortably under it) — so the choice between "fully polished, borderline" and "safely under budget" could be made with real numbers from both rather than a guess. The optimized branch was selected and merged into `main`.

Content decisions driven by an inability to visually verify results

Several passes deliberately avoided assembling art that required visual judgment to get right, where this session's tooling could not render a preview to catch a mistake before it shipped: an autotile terrain sheet (used only once a human-painted reference existed to extract from programmatically), and an animated decorative sprite (a static variant was used instead). Both are recorded inline in the affected reports rather than silently substituted. A studio with an artist in the loop and a normal build-and-look iteration cycle wouldn't hit this constraint at all — it's specific to this project's one-way, no-visual-preview development process, not a limitation of the art or the engine.

Pooling scoped to where churn actually justifies it

Object Pooling is applied where this project's actual churn pattern calls for it — the Production Menu's rows, buildings, units, and ranged projectiles — but deliberately not to the Information Panel's producible-unit icons or the one-shot death particle effect, both of which rebuild rarely (a selection change, an entity death) rather than continuously. Pooling either would mean managing pool-size guesses and reset logic for a churn rate low enough that plain `Instantiate / Destroy` is simpler and just as fast in practice. A live-service title with much higher session-long entity turnover would reasonably revisit both; for this demo's scope, applying the pattern uniformly everywhere it's *possible* rather than everywhere it's *warranted* would have been complexity for its own sake.

Housekeeping

`GameEvent / GameEventListener` (a parameterless `ScriptableObject` event channel, foundational infrastructure landed early) has no current caller — every concrete event the project ended up needing carries a payload. Flagged as dead code by the pre-submission audit and deliberately kept regardless: it has standing value as a ready template for any future signal that genuinely doesn't need one. One genuinely unused class from an earlier terrain-decoration pass (`BorderDecorationLayout` , whose one-off consumer script had already been removed) was deleted in the same audit, and a missing test file for `ProjectileFactory` was added to match its sibling factories' coverage.

6. Testing Summary

234 EditMode tests, 0 compile errors, run via Unity Test Framework in batchmode after every feature pass. Coverage is deliberately concentrated on engine-independent logic: grid math, pathfinding (including the corner-cutting edge case), the Production Menu's scroll-recycling math, combat/health state transitions, camera pan/zoom clamping, terrain-bake sizing math, and every controller's decision logic exposed as plain, explicit-input methods separate from their `Update()` loops.

Not covered by automated tests, by design, with a hand-test substituted:

- MonoBehaviour lifecycle behavior (`Awake` / `OnEnable`) and coroutine-driven behavior (unit movement, sustained attack ticking) — this machine's batchmode EditMode runner does not reliably drive Unity's Player Loop, a confirmed environment limitation, not a gap in the code under test.
- Animator-driven visuals (animation timing, transitions, sprite-outline sync).
- UI layout and visual appearance (art reskin correctness, panel banner placement).
- The actual on-screen draw-call count — verified instead with a Profiler screenshot showing 10 `SetPass` / 12 draw calls in an active-combat scene.

Every feature report that shipped hand-test-only behavior states exactly what to check and why it couldn't be automated, rather than asserting correctness without a way to verify it.

7. Engineering Assessment

What's solid. The humble-MonoBehaviour/plain-C#-logic split held for the entire project — no controller accumulated untestable decision logic in its `Update()` loop, including in the largest files (`SelectionController` , `SoldierBase`). Cross-module coupling stayed disciplined: a dependency audit late in the project found exactly one static field in the whole codebase (the brief-mandated Singleton) and confirmed every cross-module interaction goes through an event channel rather than a direct reference. The data-driven extensibility requirement (requirement 2) is real, not decorative — adding a fourth building type requires one new asset and, at most, a thin subclass; no UI or factory code branches on type.

What was corrected under real measurement rather than assumed. The draw-call budget was treated as an actual constraint to hit, not a checkbox: an architecture pass, a deferred measurement pass, and finally a branch-level A/B comparison against a real render-cost problem (Tilemap terrain) produced a verified number, not an estimate. Two placement/selection input-ordering bugs were caught by hand-testing, root-caused correctly on the second attempt once the first fix's own reasoning was shown to be racy, and the fix removed the race's precondition entirely rather than trying to make the ordering deterministic.

Known, accepted limitations. The terrain-bake load-frame spike (§5) is the clearest example: identified, measured, and left as-is with a specific, real-world justification rather than engineered away for its own sake. The lack of footprint-aware attack ranging for buildings is a scoped architectural boundary (avoiding a

circular module dependency), not an oversight — it's a one-line configuration fix (a larger `AttackRange`) if it ever proves visually wrong. Neither of these is presented as unconditionally correct; both are documented trade-offs with a stated reason and a known way to revisit them.

Where a longer-lived project would diverge. The Production Menu's pooling is architecturally real (verified by test that the pool never grows with catalog size) but only ever exercised with two catalog entries — an evaluator won't see recycling happen on screen without adding more building types first. `Health.Damaged / Died` as plain per-instance events (rather than routed through the SO event-channel system used elsewhere) is the correct choice for this project's per-instance shape, but is worth knowing as an intentional asymmetry rather than an inconsistency. The Information Panel's producible-unit icons and the one-shot death particle effect are both deliberately left unpooled (\$5, "Pooling scoped to where churn actually justifies it") because this project's churn rate doesn't justify the complexity — a live game with much higher entity turnover (hundreds of deaths per session, a constantly-rebuilding info panel) would revisit both. The project has no save/load, no resource economy, and no opposing AI — all explicitly out of scope per the brief, not gaps. None of these are things the brief graded against; they're flagged here so a reader can distinguish "cut for this demo's scope" from "the team didn't think of it."

8. Deliverables

- **Source:** this repository.
- **Windows build:** produced separately at `E:\GameBuilds\PanteonCaseProject` , confirmed working by hand-test, to be zipped and shared alongside the repository link — not committed into version control, per standard practice for built binaries.
- **This document:** `Docs/Reports/DEVELOPMENT_REPORT.md` / `Docs/Reports/DEVELOPMENT_REPORT.pdf` .